
CS 104.3 -IT Solution Architecture

Lesson 2-Instruction Set Architecture

Warm Up

Which is heavier?

- A. 1 kg of iron
- B. 1 kg of cotton
- C. They are the same



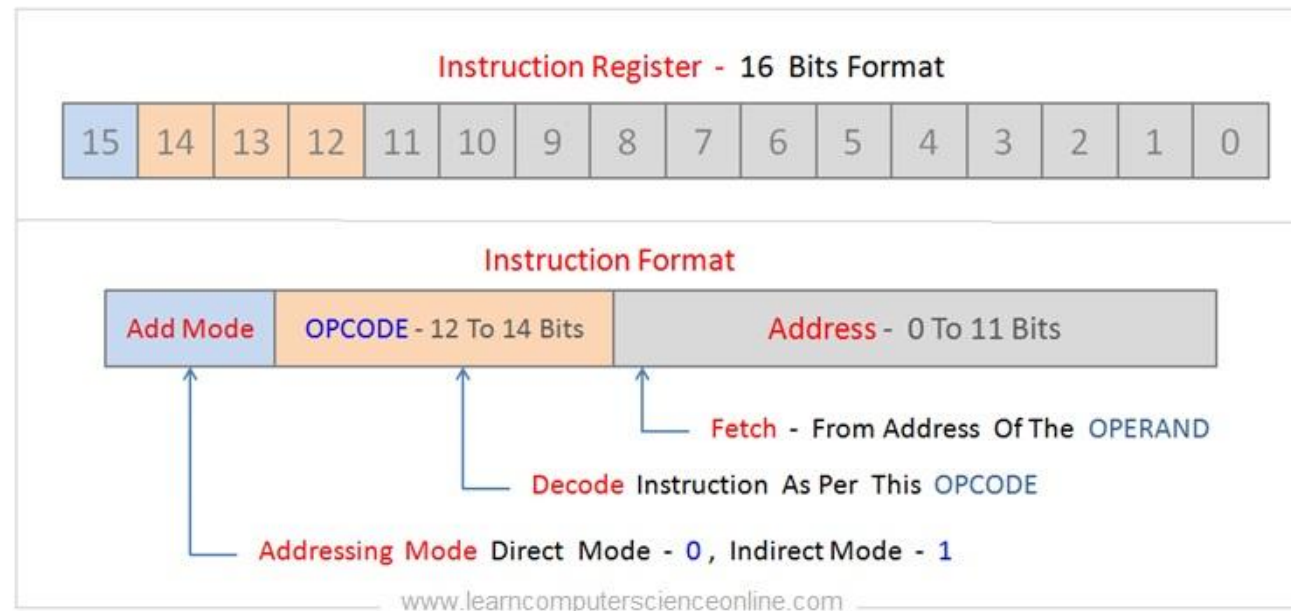
Instruction Set Architecture

- The **Instruction Set Architecture (ISA)** is the **interface between software and hardware**.
- It defines:
 - The **set of machine-level instructions** the CPU can understand and execute.
 - How each instruction is **formatted** (structure).
 - The available **addressing modes** for accessing data.
 - How memory and registers are accessed.
 - Rules for **control flow, arithmetic, logic, and data movement**.

Instruction

Definition

- An instruction is a **binary-coded command** given to the CPU that tells it what operation to perform and, when necessary, which data or operands to use.



Instruction Representation

- Computer instructions can be represented in two primary forms:
 - **Machine Language:** The binary representation (0s and 1s) directly understood by the processor.
 - Example:- 0001110010000110
 - **Assembly Language:** A human-readable symbolic representation of machine language, making programming more manageable.
 - Example :- ADD R6,R2,R6

Assembly Language

- **LC-3**

- Typically used for **introductory computer architecture/ISA teaching** because it has a small, simple instruction set.

LC-3 assembly

```
LDR  R3, R0, #2
```

- **MIPS**

- Commonly used for **computer organization and processor architecture** and is more widely used in textbooks and practical examples.
- Different varieties are there.

MIPS assembly

```
lw   $s3, 8($s0)
```

Instruction set

- An **instruction set** is the collection of instructions that a processor can understand and execute.
- Example
 - MOV
 - ADD
 - SUB
 - MUL
 - AND

Activity

- Some examples of ISA given below. Quickly search one sentence on each.
 - ARM
 - x86
 - RISC-V
 - MIPS
 - Power ISA

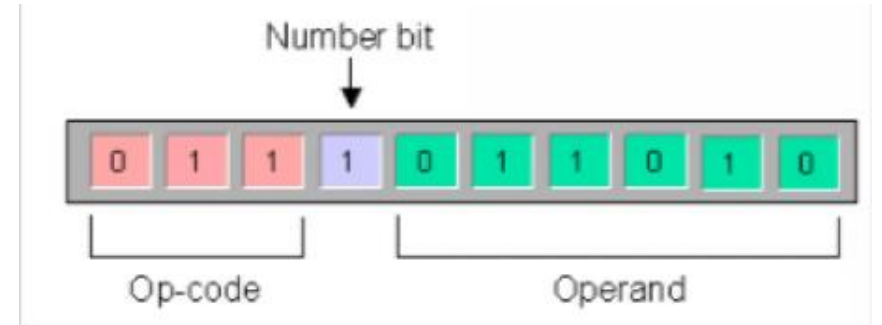
Opcodes and Operands

- **Opcode(operations code)**

- Specifies the type of operation to be performed.
- Examples: LOAD, ADD, STORE, JMP.

- **Operand**

- Provides information on the data needed for the instruction execution.
- Can be:
 - A register (e.g., A, B)
 - A memory address (e.g., 1000, 2000)
 - An immediate value (e.g., #5 in ADD A, #5)



Wake up



Memory Instructions (Load/Store)

- Example

Opcode	Generic notation	Meaning
LOAD	LOAD A, [1000]	Memory → Register
STORE	STORE A, [1000]	Register → Memory

Exercise

- If
 - Memory[1000] = 25
- Instruction: LOAD A, 1000
- What is the value in register A , after execution above instruction?

CPU Register	
A	0
R0	
--	
--	
--	

Memory	
1000	25
1001	
1002	
--	
--	

Exercise

- If the value in A register need to store in the 1000 memory location write the assembly Instruction

CPU Register	
A	200
R0	
--	
--	
--	

Memory	
1000	
1001	
1002	
--	
--	

Arithmetic Instructions

Opcode	Generic Example	Meaning
ADD	ADD A, B	Addition
SUB	SUB A, B	Subtraction
MUL	MUL A, B	Multiplication
INC	INC A	Increment
DEC	DEC A	Decrement

Exercise

- What is the final value of R3 when execute the instruction **ADD R3, R1, R2**
- When it runs the instruction **INC R3** , what will be the final value in R3

CPU Register	
A	
R0	
R1	10
R2	20
R3	0

Memory
1000
1001
1002
--
--

Exercise

- IF Register A = 2
- Trace Register A.

INC A

ADD A, A

DEC A

CPU Register

A 2

R0

--

--

--

Memory

1000

1001

1002

--

--

Exercise

- Write the assembly instructions to perform the following operations
 - Add R1 and R2 and store result in R3
 - Increment R3 by one
 - Store R3 in memory location 1000

Answer

ADD R3, R1, R2

INC R3

STORE R3, [1000]

CPU Register

A

R0

--

--

--

Memory

1000

1001

1002

--

--

Data movement and Control flow

Opcode	Example	Meaning
MOV	MOV A, B	Copy value of B into A
JMP	JMP [200]	Jump to instruction at address 200

Exercise

- **MOV A, B**
- What are the values of A, B and R0 registers after execution of above instruction.

CPU Register	
A	100
B	20
R0	0
--	
--	

Memory	
100	
104	
108	
112	
116	
120	

Exercise

- Which instruction is executed after **JMP [120]**?
- Which instructions are skipped?
- What happens to the program counter?
- What are the values of A and B when the instruction at address 120 is executed?

CPU Register	
A	0
B	0
R0	0
--	
--	

Memory	
100	MOV B, #20
104	MOV A, 310
108	JMP [120]
112	MOV A, #50
116	MOV B, #60
120	MOV A,B

Exercise

- Match the instruction opcodes with its meaning

Instruction	Meaning
ADD	A. Copy data
LOAD	B. Jump to address
MOV	C. Add values
JMP	D. Load from memory

Instruction Opcodes

Generic instruction	Meaning
LOAD	Load from memory
STORE	Store into memory
MOVE	Copy B to A
ADD	Add B to A
SUB	Subtract B from A
MUL	Multiply A by B
INC	Increment by one
DEC	Decrement by one
AND	Bitwise AND
OR	Bitwise OR
XOR	Bitwise XOR
JUMP	Jump to address
COMPARE	Compare A and B
BRANCH label	Change execution location

ARM

- ARM is a processor architecture/ISA.
- ARM instructions tell the processor what operation to perform.
- ARM includes R0-R12 general purpose registers.
- ARM architectures are widely used in:
 - Smartphones
 - Tablets
 - Embedded systems
 - IoT devices

ARM Instruction

Instruction	Full Meaning	What it Does	Example	Meaning of Example
ADD	Add	Adds two values	ADD R2, R0, R1	$R2 = R0 + R1$
SUB	Subtract	Subtracts one value from another	SUB R2, R0, R1	$R2 = R0 - R1$
MUL	Multiply	Multiplies two values	MUL R2, R0, R1	$R2 = R0 \times R1$
AND	AND	Performs bitwise AND	AND R2, R0, R1	$R2 = R0 \text{ AND } R1$
ORR	OR	Performs bitwise OR	ORR R2, R0, R1	$R2 = R0 \text{ OR } R1$
EOR	Exclusive OR (XOR)	Performs bitwise XOR	EOR R2, R0, R1	$R2 = R0 \text{ XOR } R1$
MOV	Move	Copies a value from one place to another	MOV R0, R1	$R0 = R1$
LDR	Load Register	Loads data from memory into a register	LDR R0, [R1]	$R0 = \text{Memory}[R1]$
STR	Store Register	Stores register data into memory	STR R0, [R1]	$\text{Memory}[R1] = R0$
CMP	Compare	Compares two values	CMP R0, R1	Compares R0 and R1
B	Branch	Jumps to another instruction	B LOOP	Go to LOOP
BEQ	Branch if Equal	Branches if values are equal	BEQ LOOP	Go to LOOP if equal
BNE	Branch if Not Equal	Branches if values are not equal	BNE LOOP	Go to LOOP if not equal
BL	Branch with Link	Calls a subroutine/function	BL FUNCTION	Call FUNCTION
BX	Branch and Exchange	Branches to an address, commonly used to return	BX LR	Return to address in LR

Thank You!
